

REPORT ON ROBOT OPERATING SYSTEM AND SMART CARD OPERATING SYSTEM



ADVANCED COLLEGE
OF ENGINEERING & MANAGEMENT

Submitted by:

Gaurav Sharan Kumar (ACE081BCT031)

Karun Rajbanshi (ACE081BCT035)

Kuldeep Kumar Das (ACE081BCT040)

Department of Computer and Electronics Engineering

Advance College Of Engineering and Management

Date: Shrawan 19 2083

Preface

This report has been prepared as supplementary study material to accompany a presentation on Operating System concepts as applied to two distinct domains: the Robot Operating System (ROS), a middleware framework widely used in robotics, and the Smart Card Operating System (SCOS), a specialised operating system that runs on the embedded microprocessor of a smart card.

The objective of this report is to explain, in a clear and structured manner, the core architecture, working principles, communication mechanisms, and practical examples of both systems so that it may serve as a self-contained reference for conceptual understanding.

Table of Contents

Preface	2
1. Introduction to the Robot Operating System	4
1.1 Historical Background	4
1.2 Design Goals	5
2. Conceptual Levels of ROS	5
2.1 Filesystem Level	6
2.2 Computation Graph Level	6
2.3 Community Level	7
3. Node Communication and the Publish–Subscribe Model	7
3.1 Role of the ROS Master	8
3.2 Services: Request–Response Communication	8
3.3 Parameters	9
3.4 Illustrative Example	9
4. Robot Operating System — Command Layer and Practical Tools	9
4.1 Advantages of ROS	10
4.2 Limitations of ROS	10
5. Introduction to Smart Cards	11
5.1 Types of Smart Cards	11
5.2 Typical Applications	11
6. Architecture of a Smart Card Operating System	12
6.1 Hardware Layer	13
6.2 Transmission (Communication) Layer	13
6.3 Security Manager	13
6.4 File System Manager	13
6.5 Application Interface / Card Application Layer	13
7. Application Protocol Data Units (APDU)	14
8. Smart Card File System	15
9. Worked Example: Three-Tier Access Control on a Smart Card	16
9.1 Security Features Summary	17
10. Conclusion	18
References	18

Robot and Smart Card Operating System

1. Introduction to the Robot Operating System

The Robot Operating System (ROS) is not an operating system in the traditional sense of a system that manages a computer's hardware resources such as the CPU, memory, and disk. Rather, ROS is an open-source middleware framework i.e a flexible collection of software libraries, tools, and conventions that runs on top of a conventional host operating system (typically Linux) to simplify the task of building complex and robust robot behaviour across a wide variety of robotic platforms.

ROS provides the services that an operating system traditionally provides to applications, but specialised for robotics: hardware abstraction, low-level device control, implementation of commonly used functionality, message passing between processes, and package management. Because a modern robot typically integrates many independent subsystems such as sensors, actuators, perception algorithms, planning modules, and control loops, ROS gives developers a standard way to let these subsystems communicate with one another regardless of the programming language or hardware they run on.

1.1 Historical Background

ROS originated at Stanford University and was substantially developed at Willow Garage between 2007 and 2013 as a means of providing a common software platform for robotics research. Its design philosophy was shaped by the observation that robotics researchers were repeatedly re-implementing the same low-level functionality (drivers, kinematics, mapping, navigation) instead of building on shared, reusable code. ROS addressed this by combining a communication framework with a large ecosystem of reusable packages contributed by an active community.

1.2 Design Goals

- Peer-to-peer communication between processes (nodes), avoiding a single monolithic program.
- Language independence i.e. nodes written in C++, Python, and other languages can interoperate seamlessly.
- Tools-based approach i.e. small, single-purpose command-line and graphical tools rather than one large IDE.
- Thin, reusable software i.e. code should be usable outside ROS where possible.
- Free and open-source, encouraging a collaborative global community.

2. Conceptual Levels of ROS

ROS is generally described using three interrelated conceptual levels: the Filesystem level, the Computation Graph level, and the Community level. Understanding these levels separately makes it easier to see how code, running processes, and community resources fit together.

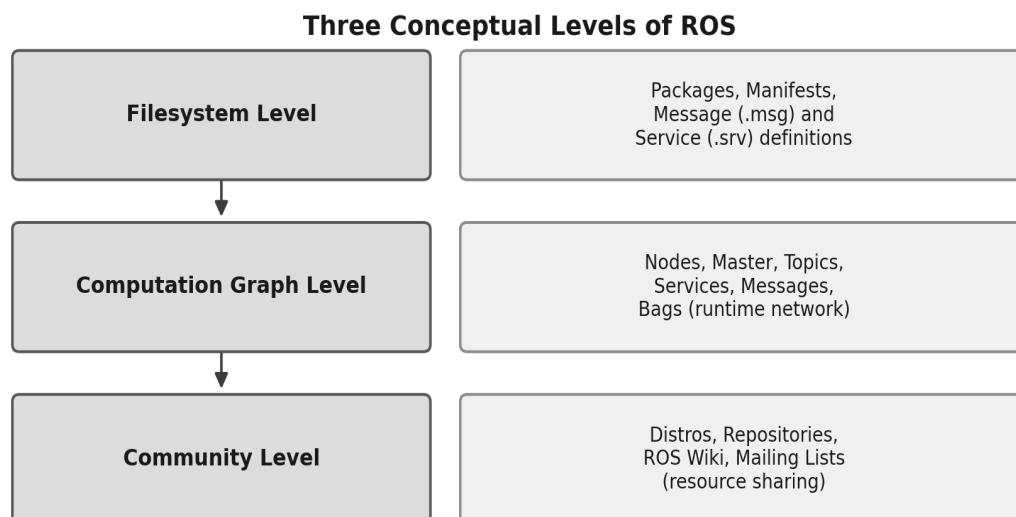


Figure 1: The three conceptual levels of ROS.

2.1 Filesystem Level

The filesystem level concerns how ROS software is organised on disk. The basic unit of organisation is the package, a directory containing source code, configuration files, and a manifest file that describes the package's dependencies and metadata.

Filesystem Concept	Description
Package	The primary unit of ROS software organisation; contains nodes, libraries, datasets and configuration.
Manifest (package.xml)	An XML file describing the package name, version, author, license, and dependencies.
Message (.msg)	A text file describing the data structure of messages passed between nodes.
Service (.srv)	A text file defining the request and response data structures for a synchronous service call.
Metapackage	A package that groups together a set of related packages for convenience.

2.2 Computation Graph Level

The computation graph is the peer-to-peer network of ROS processes that is formed at run time while all the components are executing. Every node in this network can send and receive data from other nodes through topics, services, and parameters.

Computation Graph Concept	Description
Node	A single-purpose executable process that performs computation, e.g. a sensor driver or a controller.
Master	A name service that allows nodes to locate one another; it does not route any actual data messages.

Computation Graph Concept	Description
Topic	A named bus over which nodes exchange messages using a publish/subscribe pattern.
Message	A strongly typed data structure used to exchange information over a topic.
Service	A synchronous, request/response style of communication between two nodes.
Bag	A file format used to record and play back ROS message data for testing and analysis.

2.3 Community Level

The community level consists of ROS resources that allow separate research and development groups to exchange software and knowledge, including distributions (versioned collections of compatible packages), repositories, the ROS Wiki, mailing lists, and forums.

3. Node Communication and the Publish–Subscribe Model

Nodes in ROS communicate primarily through topics using an anonymous publish/subscribe mechanism. A node that produces data (a publisher) sends messages on a named topic without needing to know which nodes, if any, will receive them. A node that requires that data (a subscriber) registers its interest in the topic and receives every message published to it. This decoupling of publishers and subscribers allows nodes to be added, removed, or restarted independently without breaking the overall system.

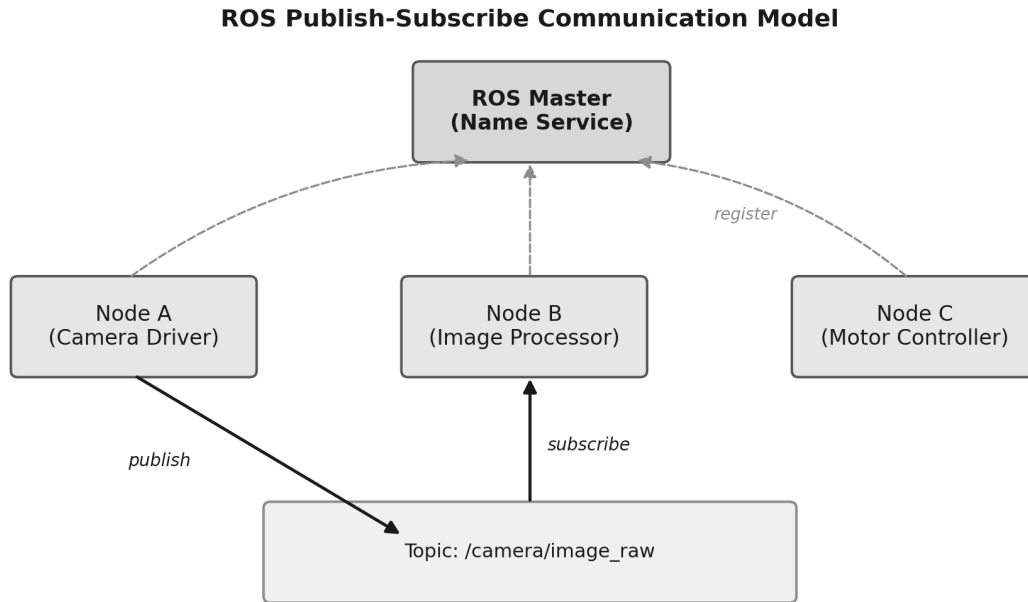


Figure 2: A camera driver node publishes image data on a topic; a processing node subscribes to it, while the ROS Master facilitates discovery between nodes.

3.1 Role of the ROS Master

The ROS Master acts purely as a name/registration service. When a node starts, it registers itself and the topics it will publish or subscribe to with the Master. When a subscriber wants data from a particular topic, the Master informs it which publisher(s) provide that topic, after which the nodes establish a direct peer-to-peer connection. Crucially, once this connection is established, message data flows directly between the nodes, the Master is never involved in the transport of the actual message payloads, which keeps the network efficient and avoids a central bottleneck.

3.2 Services: Request-Response Communication

Not all interactions fit the publish/subscribe pattern. For situations that require a synchronous request followed by a single response. For example, asking a node to perform a specific calculation and return a result, ROS provides services. A service is defined by a pair of strictly typed messages: a request message and a response

message. A client node calls the service and blocks until the response arrives, similar to a remote procedure call.

3.3 Parameters

ROS also provides a Parameter Server, a shared, multi-variate dictionary that nodes use to store and retrieve configuration values (such as calibration constants or thresholds) at run time, rather than embedding them as constants in the code.

3.4 Illustrative Example

Consider a simple mobile robot with an obstacle-avoidance behaviour built using ROS. A laser-scanner driver node publishes range readings on the topic `/scan`. An obstacle-avoidance node subscribes to `/scan`, computes a safe direction of travel, and publishes velocity commands on the topic `/cmd_vel`. A motor-controller node subscribes to `/cmd_vel` and drives the wheel motors accordingly. Each of these three nodes can be developed, tested, and restarted independently, and additional nodes (for example a logging node that also subscribes to `/scan`) can be added without modifying any existing node. This modularity is the central practical benefit of the ROS communication model.

Note: The turtlesim package, a simple 2D turtle simulator bundled with ROS, is commonly used in coursework to demonstrate nodes, topics, and services in a safe, hardware-free environment before working with a physical robot.

4. ROS-Command Layer and Practical Tools

In addition to the communication architecture, ROS provides a set of command-line tools that mirror many of the responsibilities a conventional operating system shell provides for processes and files.

Tool	Purpose
roscore	Starts the ROS Master, the parameter server, and a logging node required before any node can run.

Tool	Purpose
roslaunch	Runs a single node from a specified package.
rostopic	Starts multiple nodes and sets parameters using a single XML launch file.
rostopic	Lists, inspects, and publishes to topics from the command line.
roslaunch	Lists and inspects currently running nodes.
roslaunch	Records and replays streams of ROS message data.

4.1 Advantages of ROS

- Modular architecture allows independent development and testing of individual components.
- Large ecosystem of reusable, community-contributed packages for navigation, perception, and manipulation.
- Hardware abstraction allows the same high-level code to run on different robot platforms.
- Strong tooling for visualisation (RViz), simulation (Gazebo), and data logging (roslaunch).

4.2 Limitations of ROS

- The original ROS 1 Master represents a single point of failure for node discovery.
- Not designed for hard real-time control without additional real-time extensions.
- Network-transparent design can introduce communication latency and requires careful bandwidth management.

These limitations were significant motivating factors behind ROS 2, a redesign built on the Data Distribution Service (DDS) standard that removes the central Master and adds support for real-time and multi-robot systems.

5. Introduction to Smart card Operating System

A smart card is a pocket-sized plastic card embedded with an integrated circuit that can store and process data. Unlike a simple magnetic-stripe card, which only stores static data, a smart card containing a microprocessor can execute programs, perform cryptographic computations, and enforce access control over the data it stores. The software that manages this microprocessor, its memory, and its communication with the outside world is called the Smart Card Operating System (SCOS), sometimes referred to as the Card Operating System (COS).

5.1 Types of Smart Cards

Type	Description
Memory card	Contains only memory and fixed logic; offers storage but little or no processing capability.
Microprocessor card	Contains a CPU, ROM, EEPROM/Flash and RAM, capable of running an operating system and applications.
Contact card	Communicates through physical electrical contacts on the card surface (ISO/IEC 7816).
Contactless card	Communicates via radio frequency induction using an embedded antenna (ISO/IEC 14443).
Dual-interface card	Supports both contact and contactless communication on a single chip.

5.2 Typical Applications

- Banking and payment cards (EMV chip cards).
- Subscriber Identity Module (SIM) cards used in mobile telephony.
- National identity, healthcare, and passport cards.
- Physical access-control and time-attendance cards.
- Transit and loyalty cards.

6. Architecture of a Smart Card Operating System

A smart card operating system is deliberately minimal compared to a desktop operating system, since it must run within extremely limited memory and processing resources while providing very strong guarantees of security. It is typically organised as a set of layers, each responsible for a distinct concern, sitting on top of the physical microprocessor hardware.

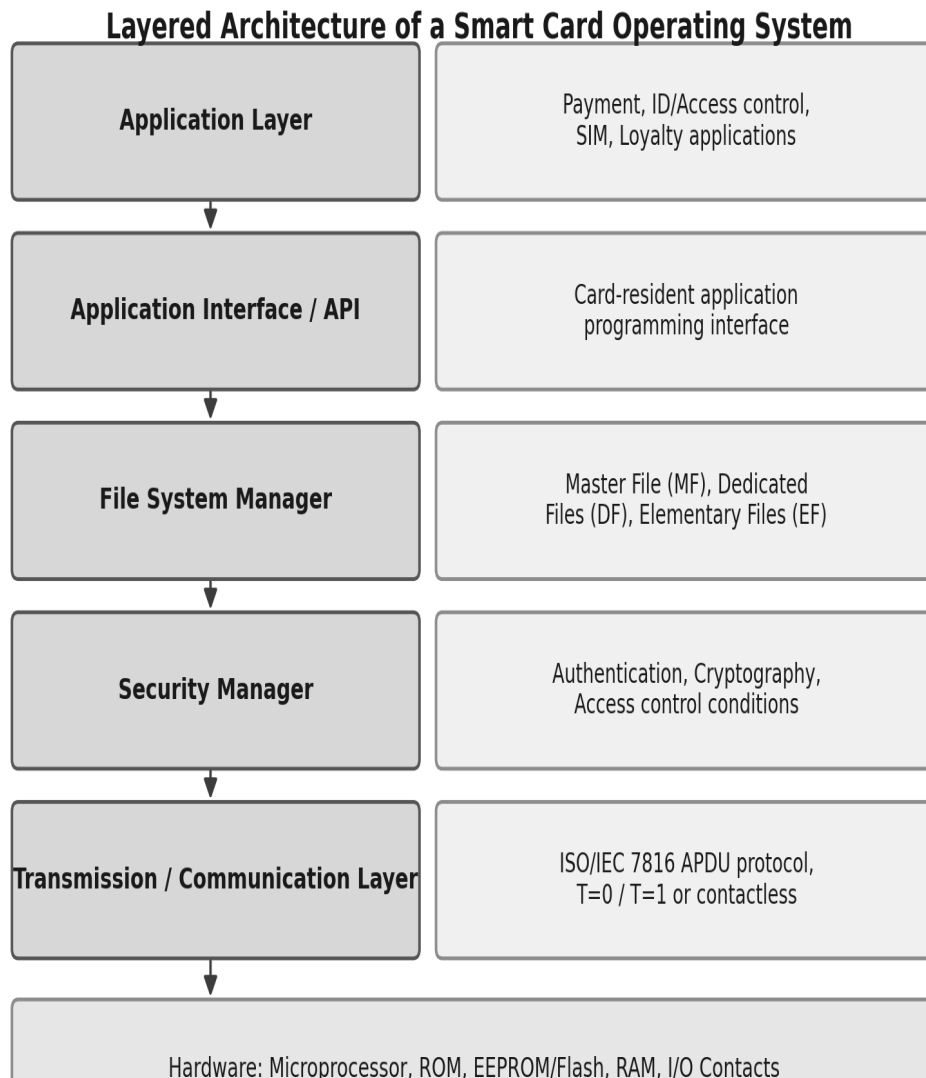


Figure 3: A typical layered architecture of a smart card operating system.

6.1 Hardware Layer

At the base of the system is the physical chip: an 8-bit, 16-bit, or 32-bit microprocessor, a small amount of ROM (holding the fixed operating system code), EEPROM or Flash memory (holding the file system and application data, which must persist when the card is removed from power), a small RAM for temporary working data, and an input/output interface either the physical electrical contacts defined by ISO/IEC 7816 or a radio antenna for contactless operation.

6.2 Transmission (Communication) Layer

This layer implements the low-level protocol used to exchange data between the card and a card-accepting device known as the Card Acceptance Device or terminal reader. For contact cards this is governed by the ISO/IEC 7816-3 standard, which defines the T=0 (byte-oriented) and T=1 (block-oriented) transmission protocols. All communication at this level is carried out using Application Protocol Data Units (APDUs), discussed in detail in Section 7.

6.3 Security Manager

The security manager enforces authentication and access-control policies before allowing any command to read, write, or execute data held on the card. It typically supports PIN verification, mutual cryptographic authentication between the card and the terminal, and enforcement of access conditions attached to individual files, such as 'always', 'never', 'PIN required', or 'secure-messaging required'.

6.4 File System Manager

The file system manager organises persistent data on the card using a strict hierarchical structure, described fully in Section 8.

6.5 Application Interface / Card Application Layer

At the top of the stack sit one or more card applications, for example: a banking application or a transit-fare application which use the services offered by the lower layers through a well-defined Application Programming Interface (API),

often standardised by frameworks such as Java Card or MULTOS, which allow multiple independent applications to reside securely on the same physical card.

7. Application Protocol Data Units (APDU)

All communication between a smart card and a terminal takes place through a simple, well-defined message exchange called an Application Protocol Data Unit, standardised in ISO/IEC 7816-4. Every transaction consists of exactly one Command APDU sent from the terminal to the card and exactly one Response APDU sent back.

Command APDU and Response APDU Structure

Command APDU:



Response APDU:

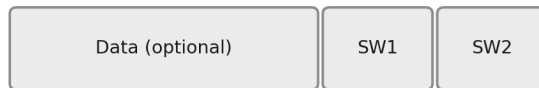


Figure 4: Structure of a Command APDU and a Response APDU.

Field	Meaning
CLA	Class byte identifies the category of the command (e.g. ISO-standard or proprietary).
INS	Instruction byte specifies the actual operation to perform (e.g. SELECT FILE, VERIFY, READ BINARY).
P1, P2	Parameter bytes provide additional qualifiers for the instruction.
Lc	Length of the command data field that follows.

Field	Meaning
Data	The optional command data being sent to the card.
Le	Expected length of the data to be returned in the response.
SW1, SW2	Status word bytes returned by the card indicating success or the specific reason for failure (e.g. 90 00 = success).

For example, to read a user's stored balance from a transit card, the terminal would first send a SELECT FILE command APDU to choose the correct elementary file, then send a READ BINARY command APDU to retrieve its contents; the card responds to each with a Response APDU containing the requested data (if any) followed by a two-byte status word.

8. Smart Card File System

Persistent data on a smart card is organised as a strict tree-structured file system defined by ISO/IEC 7816-4, conceptually similar to directories and files in a conventional operating system but with much stricter access rules.

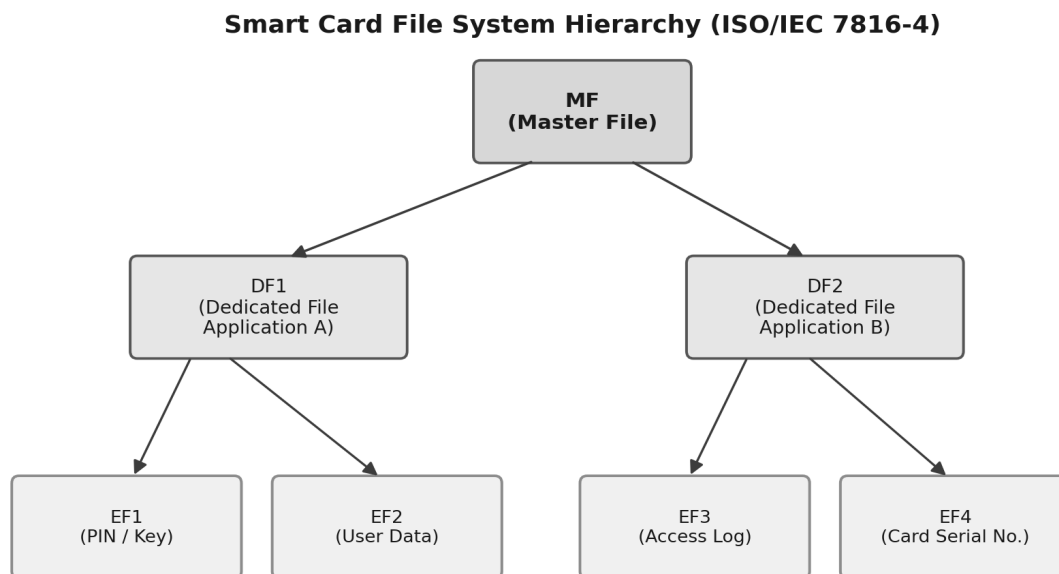


Figure 5: Hierarchical smart card file system: Master File, Dedicated Files, and Elementary Files.

Element	Description
Master File (MF)	The single root directory of the card's file system, analogous to the root directory of a disk.
Dedicated File (DF)	An intermediate directory-like node, often used to group the files belonging to one application.
Elementary File (EF)	A leaf node that actually stores data . For example a PIN, a cryptographic key, or user records.

Every file on the card carries its own access conditions, which the security manager checks against the current authentication state before permitting any operation. For instance, an elementary file holding a cryptographic key might be marked as never readable, while a file holding a transaction log might be readable by anyone but writable only after a successful PIN verification or mutual authentication.

9. Worked Example: 3Tier Access Control on a Smart Card

To illustrate how these concepts combine in a real system, consider an access-control card used to gate command execution on an embedded device (such as a mobile robot) with three privilege levels: Guest, Operator, and Admin.

1. The card is inserted or tapped at the reader, and the terminal (microcontroller) sends a SELECT FILE command APDU to select the application's Dedicated File.
2. The terminal issues a READ BINARY or GET DATA command APDU to retrieve the card's identifier and the stored role level (Guest, Operator, or Admin) from the corresponding Elementary File.
3. If the requested operation requires elevated privileges, the security manager on the card demands PIN verification or a cryptographic challenge-response authentication before releasing the associated data or permitting the command.
4. The card's operating system checks the access condition attached to the requested file or operation against the current authentication state and

returns an appropriate status word 90 00 for success, or an error status word such as 69 82 (security status not satisfied) if the access condition is not met.

5. Only after a successful response does the host device execute the command corresponding to the verified privilege level, for example allowing an Admin-level card to issue configuration commands that a Guest-level card cannot.

This pattern select, authenticate, verify access condition, respond is the same general sequence used by banking, transit, and identity applications, which is why the smart card file system and APDU protocol described above form the common foundation for a wide range of real-world secure applications.

9.1 Security Features Summary

- PIN / password verification (something the user knows).
- Cryptographic authentication and digital signatures (something the card proves it has, e.g. a secret key).
- Access conditions attached to individual files and operations.
- Secure messaging through encryption and integrity protection of APDU data in transit.
- Tamper-resistant hardware design that resists physical and side-channel attacks.

10. Conclusion

Although the Robot Operating System and the Smart Card Operating System serve entirely different purposes, studying them together highlights the breadth of the term 'operating system' in modern computing. ROS is a distributed, network-transparent middleware that coordinates many cooperating processes across a robot's sensors and actuators using a publish/subscribe communication model, prioritising modularity and code reuse. The Smart Card Operating System, by contrast, is a highly constrained, security-focused system that manages a tiny amount of processing power and memory inside a single tamper-resistant chip, prioritising strict access control and cryptographic integrity above all else.

Both systems nonetheless address the same fundamental operating-system concerns such as process/node management, inter-process communication, resource and memory management, file systems, and security and access control but apply very different design trade-offs suited to their respective environments.

References

1. Quigley, M., Gerkey, B., & Smart, W. D. (2015). Programming Robots with ROS. O'Reilly Media.
2. Open Robotics. ROS Wiki and Documentation - wiki.ros.org.
3. ISO/IEC 7816-3: Identification cards - Integrated circuit cards, Part 3: Cards with contacts — Electrical interface and transmission protocols.
4. ISO/IEC 7816-4: Identification cards - Integrated circuit cards, Part 4: Organization, security and commands for interchange.
5. Rankl, W., & Effing, W. (2010). Smart Card Handbook (4th ed.). Wiley.
6. Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts. Wiley.